

Yesterday 2:47 PM

servelets

≡ ChatGPT   Upgrade

Fast answer

It looks like you meant **Servlets** (not "Servelets").

A **Servlet** is a Java program that runs on a web server and handles client requests (usually HTTP requests) to generate dynamic web content.

How Servlets Work

1. A client (browser) sends an HTTP request.
2. The web server forwards the request to the servlet container (such as Apache Tomcat).
3. The servlet processes the request.
4. It interacts with databases or business logic if needed.
5. The servlet sends an HTTP response back to the client.

Servlet Lifecycle

The lifecycle of a servlet consists of three main methods:

1. **init()**
 - Called once when the servlet is loaded.
 - Used for initialization.
2. **service()**
 - Called for every client request.
 - Dispatches requests to methods like `doGet()` or `doPost()`.
3. **destroy()**
 - Called once before the servlet is removed.
 - Used to release resources.

Example Servlet

 Java



```
import java.io.*;
import jakarta.servlet.*;
```

```
import jakarta.servlet.http.*;

public class HelloServlet extends HttpServlet {

    public void doGet(HttpServletRequest request,
                      HttpServletResponse response)
        throws ServletException, IOException {

        response.setContentType("text/html");

        PrintWriter out = response.getWriter();
        out.println("<h1>Hello, Welcome to Servlets!</h1>");

    }
}
```

Advantages of Servlets

- Platform independent (Java-based)
- Better performance than CGI (single instance handles multiple requests)
- Secure and robust
- Supports session management
- Easy integration with databases and other Java technologies

Common Servlet Methods

- `doGet()` – Handles HTTP GET requests.
- `doPost()` – Handles HTTP POST requests.
- `doPut()` – Handles HTTP PUT requests.
- `doDelete()` – Handles HTTP DELETE requests.

Servlets are the foundation of many Java web applications and are commonly used with technologies such as JSP, Spring MVC, and Spring Boot.

Yesterday 2:49 PM

Servlets: Dynamic Content Generation

Yesterday 5:08 PM

Today 2:49 AM

Session in Servlet

What is a Session?

A **Session** is a mechanism used to maintain user-specific data between multiple requests.

Since HTTP is a **stateless protocol**, the server does not remember previous requests. Session management helps identify the same user across multiple requests.

Example

User Login

|

v

Request 1 --> Server

|

v

Request 2 --> Server

|

v

Request 3 --> Server



Using a session, the server knows all requests belong to the same user.

Session Management

Definition

Session Management is the process of storing and maintaining user information across multiple requests.

Information Stored in Session

- User ID
- Username
- Shopping Cart
- Login Status
- Preferences

Why Session Management?

HTTP is stateless:

Request 1 --> Server

Request 2 --> Server

Request 3 --> Server



The server treats each request independently.

Session management creates continuity.

Session Tracking Techniques

Servlets provide four session tracking techniques:

1. Cookies

2. HttpSession
3. URL Rewriting
4. Hidden Form Fields

Most Important for Exams: Cookies and HttpSession

Session Tracking with Cookies

What is a Cookie?

A **Cookie** is a small text file stored in the client's browser.

The server sends cookies to the browser, and the browser sends them back with future requests.

Creating a Cookie

</> Java



```
Cookie c =  
    new Cookie("username", "Gaurav");  
  
response.addCookie(c);
```

Reading a Cookie

</> Java



```
Cookie cookies[] =  
    request.getCookies();  
  
for(Cookie c : cookies)  
{  
    System.out.println(  
        c.getName() + " : " +  
        c.getValue());  
}
```

Flow

```
Server  
|  
Create Cookie  
|  
Browser Stores Cookie  
|
```



Future Request
|
Cookie Sent Back

Advantages

- Easy to use
- Stored on client side
- Supports session tracking

Disadvantages

- User can disable cookies
- Limited storage size
- Less secure for sensitive data

HttpSession

What is HttpSession?

`HttpSession` is an interface used to store session data on the server side.

Package:

</> Java



`jakarta.servlet.http.HttpSession`

It is the most commonly used session tracking technique.

Creating a Session

</> Java



```
HttpSession session =  
    request.getSession();
```

If a session does not exist, a new one is created.

Storing Data in Session

</> Java



```
session.setAttribute(  
    "username",  
    "Gaurav");
```

Retrieving Data

</> Java



```
String name =  
(String)session.getAttribute(  
    "username");
```

Removing an Attribute

</> Java



```
session.removeAttribute(  
    "username");
```

Invalidating a Session

</> Java



```
session.invalidate();
```

Used during logout.

Complete Example

Login Servlet

</> Java



```
import jakarta.servlet.http.*;  
  
public class LoginServlet  
    extends HttpServlet {
```

```
protected void doPost(  
    HttpServletRequest req,  
    HttpServletResponse res) {  
  
    HttpSession session =  
        req.getSession();  
  
    session.setAttribute(  
        "user",  
        "Gaurav");  
    }  
}
```

Profile Servlet

</> Java



```
HttpSession session =  
    request.getSession(false);  
  
String user =  
(String)session.getAttribute(  
    "user");  
  
response.getWriter()  
    .println("Welcome "  
        + user);
```

Output

Welcome Gaurav



Important HttpSession Methods

Method	Purpose
getSession()	Create/Get session
setAttribute()	Store data
getAttribute()	Retrieve data
removeAttribute()	Remove data

Method	Purpose
invalidate()	Destroy session
getId()	Get session ID
isNew()	Check if session is new

Cookies vs HttpSession

Cookies	HttpSession
Stored on client	Stored on server
Less secure	More secure
Small storage	Larger storage
User can disable	Managed by server
Browser dependent	Server dependent

Session Life Cycle

User Login

|

v

Session Created

|

v

Store Attributes

|

v

Multiple Requests

|

v

Logout

|

v

Session Invalidated



MCQ Points

1. HTTP is a **stateless protocol**.
2. Session management maintains user data across requests.
3. A cookie is stored on the **client side**.
4. HttpSession stores data on the **server side**.
5. `request.getSession()` creates or returns a session.
6. `setAttribute()` stores data in a session.
7. `getAttribute()` retrieves session data.
8. `invalidate()` destroys a session.
9. Cookies can be disabled by the browser.
10. HttpSession is more secure than cookies.

Exam Definition

Session: A session is a mechanism that maintains user-specific information across multiple HTTP requests.

Session Management: The process of preserving user data and state between requests in a web application.

Cookies: Small text files stored on the client browser for tracking user information.

HttpSession: A server-side session tracking mechanism that stores and manages user data across multiple requests.

Request Dispatcher

RequestDispatcher in Servlet

What is RequestDispatcher?

RequestDispatcher is an interface used to transfer a request from one resource to another resource on the server.

Resources can be:

- Servlet
- JSP
- HTML page

Package:

</> Java



```
jakarta.servlet.RequestDispatcher
```

It is mainly used for:

1. Forwarding a request
2. Including content from another resource

Methods of RequestDispatcher

Method	Purpose
forward()	Transfers request to another resource
include()	Includes output of another resource

1. forward() Method

The `forward()` method transfers control from one servlet to another resource.

Syntax

</> Java



```
RequestDispatcher rd =  
request.getRequestDispatcher("second");  
  
rd.forward(request, response);
```

Example

First Servlet

</> Java



```
import jakarta.servlet.*;  
import jakarta.servlet.http.*;  
import java.io.*;  
  
public class FirstServlet extends HttpServlet {  
  
    protected void doGet(  
        HttpServletRequest request,
```

```

        HttpServletResponse response)
        throws ServletException, IOException {

    RequestDispatcher rd =
        request.getRequestDispatcher(
            "SecondServlet");

    rd.forward(request, response);
    }
}

```

Second Servlet

</> Java



```

public class SecondServlet
    extends HttpServlet {

    protected void doGet(
        HttpServletRequest request,
        HttpServletResponse response)
        throws IOException {

        response.getWriter()
            .println(
                "Welcome to Second Servlet");
    }
}

```

Flow

```

Browser
|
v
FirstServlet
|
forward()
|
v
SecondServlet
|
v
Response

```



Important Points

- Request object remains the same.
- Response object remains the same.
- URL in browser does not change.

- Control completely moves to the target resource.
-

2. include() Method

The `include()` method includes the output of another resource in the current response.

Syntax

</> Java



```
RequestDispatcher rd =  
request.getRequestDispatcher("second");  
  
rd.include(request, response);
```

Example

</> Java



```
import jakarta.servlet.*;  
import jakarta.servlet.http.*;  
import java.io.*;  
  
public class FirstServlet  
    extends HttpServlet {  
  
    protected void doGet(  
        HttpServletRequest request,  
        HttpServletResponse response)  
        throws IOException,  
            ServletException {  
  
        PrintWriter out =  
            response.getWriter();  
  
        out.println("Content of First Servlet");  
  
        RequestDispatcher rd =  
            request.getRequestDispatcher(  
                "SecondServlet");  
  
        rd.include(request, response);  
    }  
}
```

Second Servlet

</> Java



```
public class SecondServlet
    extends HttpServlet {

    protected void doGet(
        HttpServletRequest request,
        HttpServletResponse response)
        throws IOException {

        response.getWriter()
            .println(
                "Content of Second Servlet");
    }
}
```

Output

Content of First Servlet
Content of Second Servlet



Flow

```
Browser
|
v
FirstServlet
|
include()
|
v
SecondServlet
|
v
Combined Response
```



Difference Between forward() and include()

forward()	include()
Transfers control completely	Includes output only
Original servlet stops execution	Original servlet continues
Generates one final response	Combines multiple outputs

forward()	include()
Used for navigation	Used for reusable content
Browser URL remains unchanged	Browser URL remains unchanged

How to Obtain RequestDispatcher

</> Java



```
RequestDispatcher rd =  
request.getRequestDispatcher(  
    "targetServlet");
```

or

</> Java



```
RequestDispatcher rd =  
getServletContext()  
.getRequestDispatcher(  
    "/targetServlet");
```

Real-Life Example

Login Application

Login Form

|

v

LoginServlet

|

+----- Valid User -----> HomeServlet

|

+----- Invalid User --> Error.jsp



Using forward() :

</> Java



```
if(valid)  
{
```

```
        rd.forward(request,response);
    }
    else
    {
        rd.forward(request,response);
    }
}
```

MCQ Points

1. `RequestDispatcher` is an interface.
2. Package: `jakarta.servlet` .
3. Used for communication between server resources.
4. `forward()` transfers control to another resource.
5. `include()` includes output from another resource.
6. Browser URL does not change in `forward()` .
7. Browser URL does not change in `include()` .
8. Request and Response objects are shared.
9. `request.getRequestDispatcher()` returns a `RequestDispatcher` .
10. `forward()` is commonly used after login validation.

Exam Definition

RequestDispatcher is an interface provided by the Servlet API that allows a servlet to forward a request to another server resource or include the content of another resource in the current response using the `forward()` and `include()` methods.

Page Navigation

Page Navigation in Servlets

What is Page Navigation?

Page Navigation is the process of moving a user from one web page or resource to another after processing a request.

In Servlets, page navigation is commonly done using:

1. `RequestDispatcher.forward()`
2. `HttpServletResponse.sendRedirect()`

1. Navigation Using `RequestDispatcher.forward()`

The request is forwarded internally from one servlet/JSP to another resource.

Example

</> Java



```
RequestDispatcher rd =  
request.getRequestDispatcher("home.jsp");  
  
rd.forward(request, response);
```

Flow

```
Browser  
|  
v  
Servlet  
|  
forward()  
|  
v  
home.jsp
```



Characteristics

- Server-side navigation
- Same request and response objects
- Browser URL does not change
- Faster than redirect

2. Navigation Using `sendRedirect()`

The server instructs the browser to send a new request to another page.

Example

</> Java



```
response.sendRedirect("home.jsp");
```

Flow

Browser
|
Request
|
Servlet
|
sendRedirect()
|
Response
|
New Request
|
home.jsp



Characteristics

- Client-side navigation
- New request is created
- Browser URL changes
- Slightly slower due to an extra request

Login Example

Using forward()

</> Java



```
if(validUser)
{
    RequestDispatcher rd =
    request.getRequestDispatcher(
        "home.jsp");

    rd.forward(request,response);
}
```

Using sendRedirect()

</> Java

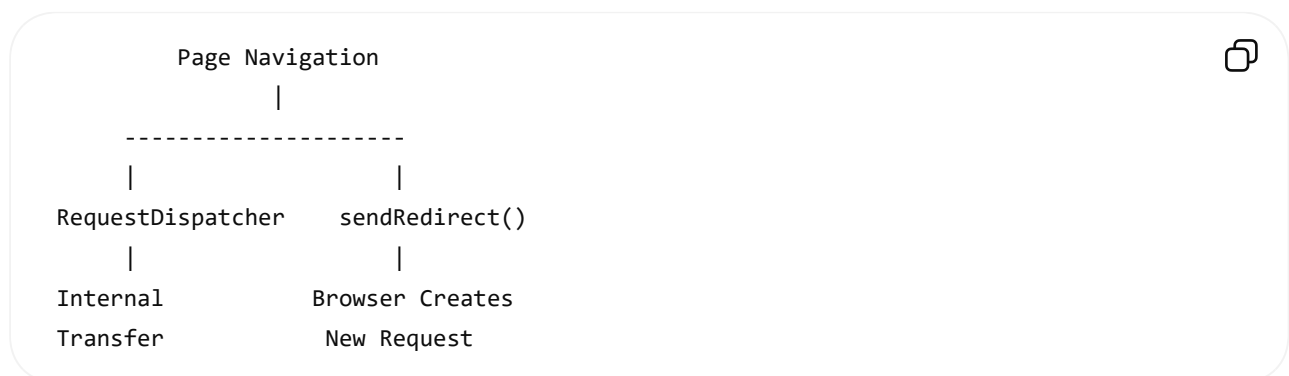


```
if(validUser)
{
    response.sendRedirect(
        "home.jsp");
}
```

Difference Between forward() and sendRedirect()

forward()	sendRedirect()
Server-side	Client-side
Same request object	New request object
URL unchanged	URL changes
Faster	Slower
Uses RequestDispatcher	Uses HttpServletResponse
Data available in request	Request data lost

Navigation Diagram



When to Use

Use forward()

- Passing request data to another page
- Internal navigation within the application
- Better performance

Use sendRedirect()

- Redirecting to another application or website
- Preventing form resubmission
- Navigating after login/logout

Example:

 Java



```
response.sendRedirect(  
    "https://www.google.com");
```

MCQ Points

1. Page navigation moves users between web resources.
2. `forward()` uses `RequestDispatcher`.
3. `sendRedirect()` uses `HttpServletResponse`.
4. `forward()` is server-side navigation.
5. `sendRedirect()` is client-side navigation.
6. URL remains unchanged in `forward()`.
7. URL changes in `sendRedirect()`.
8. `forward()` is faster than `sendRedirect()`.
9. `sendRedirect()` creates a new request.
10. Request data is preserved in `forward()` but not in `sendRedirect()`.

Exam Definition

Page Navigation in Servlets is the mechanism of transferring a user from one resource to another using techniques such as `RequestDispatcher.forward()` and `HttpServletResponse.sendRedirect()` to control the flow of a web application.